

AI / ML / GenAI

User Manual - How to Run and Use

Project: AI_ML_GENAI

Getting Started

This guide walks you through setting up the AI / ML / GenAI workspace from scratch.

Prerequisites

Requirement Details

Step 1: Install uv

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Restart your terminal or add uv to PATH:

```
$env:Path = "$env:USERPROFILE\.local\bin;$env:Path"
```

Step 2: Clone / open project

```
cd C:\MY_SPACE\AI_ML_GENAI
```

Step 3: Install dependencies

```
uv sync --extra dev
```

This creates .venv/ and installs all core packages.

Optional - local GGUF models:

```
uv sync --extra local-llm
```

Requires C++ build tools on Windows. Skip this if you use an API LLM.

Step 4: Configure environment

```
copy .env.example .env
```

Edit .env for your setup. See Configuration Reference.

Minimum for API usage:

```
USE_API_LLM=true
```

Step 5: Add documents (for RAG)

Place files in data/raw/:

* PDF (.pdf)

* Tex

Step 6: Build vector index

```
uv run python ingest.py
```

Output example: Indexed 142 chunks into vector_db

Step 7: Run the application

```
uv run streamlit run app.py
```

Open the URL shown in the terminal (usually <http://localhost:8501>).

Step 8: Verify installation

```
uv run pytest
```

All tests should pass.

Run, activate, and add dependencies

For daily use - activating .venv, running all commands, and adding packages with uv - see Run, Activate & Dependencies.

Quick reference:

```
# Activate (optional - uv run works without activation)
```

.\.venv

What to do next

Goal Read

How to Run, Activate, and Manage Dependencies

This guide covers daily development workflow: activating the environment, running the project, and adding new packages with uv.

Table of contents

1. Activate the virtual environment

2. How

Activate the virtual environment

This project uses uv and a virtual environment at .venv/ (created automatically on first uv sync).

You have two options - Option A is recommended because you do not need to activate manually.

Option A: Use uv run (recommended - no activation needed)

uv run automatically uses the project .venv for any command:

```
cd C:\MY_SPACE\AI_ML_GENAI
```

No activate step required.

Option B: Activate .venv manually

Use this if you prefer a traditional workflow or run many commands in the same shell session.

Windows PowerShell:

```
cd C:\MY_SPACE\AI_ML_GENAI
```

.\.venv

Windows Command Prompt (cmd):

```
cd C:\MY_SPACE\AI_ML_GENAI
```

.venv\

Linux / macOS:

```
cd /path/to/AI_ML_GENAI
```

source

Verify activation - your prompt should show (.venv) and Python should point to the venv:

```
python --version
```

where

Deactivate when done:

```
deactivate
```

If **.venv** does not exist yet

Create it and install dependencies first:

```
cd C:\MY_SPACE\AI_ML_GENAI
```

How to run the project

Always start from the project root:

```
cd C:\MY_SPACE\AI_ML_GENAI
```

First-time / full setup run

```
# 1. Install dependencies (creates .venv)
```

Open the browser URL from the terminal (usually <http://localhost:8501>).

Run commands reference

What to run	Command	Notes
-------------	---------	-------

Run with activated venv (Option B)

If you activated **.venv** manually, drop the **uv run** prefix:

```
.\.venv\Scripts\Activate.ps1
```

Optional: local GGUF model support

```
uv sync --extra local-llm
```

Then place a .gguf file in models/ and set LLM_MODEL_PATH in .env.

Stop the application

* Streamlit / Jupyter: Press Ctrl + C in the terminal

* MC

Add new dependencies

Dependencies are managed in pyproject.toml using uv (not pip install directly).

Add a runtime dependency

```
cd C:\MY_SPACE\AI_ML_GENAI
```

uv ad

Examples:

```
uv add fastapi          # Add REST API framework
```

uv ad

This updates pyproject.toml and uv.lock, and installs into .venv.

Add a development-only dependency

```
uv add --dev ruff        # Linter
```

uv ad

Dev dependencies go under [project.optional-dependencies] or uv's dev group.

Add an optional dependency group

For optional features (like local-llm), edit pyproject.toml or use:

```
uv add --optional local-llm llama-cpp-python
```

Then install with:

```
uv sync --extra local-llm
```

Install all dependencies after pulling changes

If someone else added packages or you cloned the repo:

```
uv sync --extra dev
```

Install with optional extras:

```
uv sync --extra dev --extra local-llm
```

Remove a dependency

```
uv remove <package-name>
```

Upgrade a dependency

```
uv add <package-name> --upgrade
```

List installed packages

```
uv pip list
```

Or with activated venv:

```
pip list
```

Important: do not use plain pip install for project deps

Avoid Use instead

Using uv add keeps pyproject.toml and uv.lock in sync for reproducible installs.

Daily workflow cheat sheet

```
# Open project
```

```
cd C:\
```

Related docs

* Getting Started - first-time setup

* Us

User Manual

Complete guide for using the AI / ML / GenAI workspace.

Table of contents

1. Application overview
2. Mo

Application overview

The main entry point is app.py, a Streamlit web UI with four modes selectable from the sidebar:

Mode	Module	Best for

Application modes flow

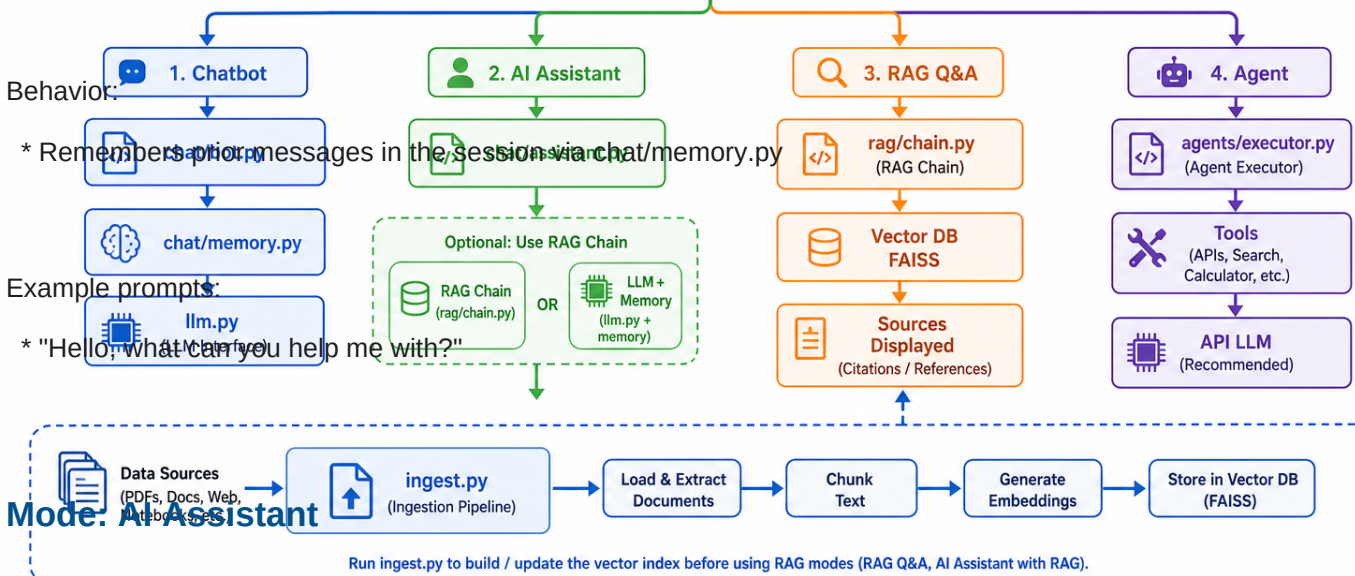
Application Modes - app.py Streamlit UI

Mode: Chatbot

Purpose: Simple conversational chat with conversation history.

How to use:

1. Run `uv run streamlit run app.py`



Mode: AI Assistant

Purpose: General-purpose assistant that can optionally ground answers in your documents.

How to use:

1. Select AI Assistant in the sidebar

Behavior:

- * With RAG enabled: searches vector_db/ and answers from retrieved context

When to use:

- * General help with optional document grounding

Mode: RAG Q&A

Purpose: Answer questions only from documents in data/raw/.

Prerequisites:

1. Documents in data/raw/

How to use:

1. Select RAG Q&A in the sidebar

Behavior:

- * Retrieves top-K relevant chunks (TOP_K in .env, default 4)

Example workflow:

```
# 1. Add report.pdf to data/raw/
```

Mode: Agent

Purpose: Run an AI agent that can call tools to complete tasks.

Prerequisites:

- * Set USE_API_LLM=true and OPENAI_API_KEY in .env (recommended)

How to use:

1. Select Agent in the sidebar

Available tools (agents/tools.py):

Tool	Description
------	-------------

Multi-step workflows (agents/workflows.py):

```
from agents.workflows import research_and_summarize
```

Document ingestion

Script: ingest.py

Pipeline:

1. Load documents from data/raw/ (rag/loader.py)

```
uv run python ingest.py
```

Re-run after:

- * Adding new documents

Supported formats: PDF, TXT, Markdown

MCP server

Purpose: Expose project tools to external clients (Cursor, Claude Desktop, etc.) via Model Context Protocol.

Start server:

```
uv run python mcp_server/server.py
```

Built-in tools:

Tool	Description
------	-------------

Cursor configuration - reference `mcp_server/config.json`:

 $\{$

Add custom tools: Register new functions in `mcp_server/server.py` or add helpers in `mcp_server/tools/`.

Notebooks

Location: notebooks/experiments.ipynb

Use for:

- * Testing chunk sizes before full ingestion

```
uv run jupyter lab notebooks/
```

Models

See Models Guide for:

- * OpenAI API vs local GGUF

Command reference

Full run/activate/dependency guide: Run, Activate & Dependencies

Activate virtual environment (optional)

```
..\env\Scripts\Activate.ps1 # PowerShell
```

Or use uv run before any command - no activation needed.

Run commands

Command	Description
---------	-------------

Setup & dependencies

Command	Description
---------	-------------

Health Check & Setup Status

Last verified: July 2026 on the development laptop (Windows 11, Intel Core Ultra 5 135H, 32 GB RAM).
Use this page to confirm the project is working and to finish setup for chat, RAG answers, and agents.

Quick status

Area	Status
------	--------

What is working

Component	Details
-----------	---------

Verify yourself

```
cd C:\MY_SPACE\AI_ML_GENAI
```

Expected: tests pass, ingest indexes chunks, MCP prints ['greet', 'word_count'].

What you still need to configure

The LLM is not ready until you choose one option below. Without this, Chatbot, AI Assistant, RAG Q&A, and Agent modes cannot generate answers.

Item	Current state	Action
------	---------------	--------

Option A - OpenAI API (fastest to test)

Best for quick testing and Agent mode.

1. Edit .env:

```
USE_API_LLM=true
```

2. Run the app:

```
uv run streamlit run app.py
```

No local model download required.

Option B - Local GGUF (this laptop)

Best for offline use. On this laptop, prefer 3B Q4_K_M for speed. See Models - this laptop profile.

1. Install local LLM support:

```
uv sync --extra local-llm
```

2. Download a model (example - fast 3B):

```
uv add huggingface-hub
```

3. Edit .env:

```
USE_API_LLM=false
```

4. Build index (if not done) and run:

```
uv run python ingest.py
```

How to run now

Always from project root:

```
cd C:\MY_SPACE\AI_ML_GENAI
```

Task Command

Open the Streamlit URL in your browser (usually `http://localhost:8501`).

After LLM is configured

Mode Expected behavior

Fixes applied during health check

These issues were found and fixed in the codebase:

Issue Fix

MCP config path: `mcp_server/config.json`

Minor warnings (non-blocking)

Warning Impact Optional fix

Full verification checklist

`cd C:\MY_SPACE\AI_ML_GENAI`

Related docs

* Run, Activate & Dependencies

* Mo

REST API (Swagger)

FastAPI REST API with interactive Swagger UI for all major features.

Start the API server

```
cd C:\MY_SPACE\AI_ML_GENAI
```

Swagger UI

Open in your browser:

URL	Description
-----	-------------

Use Swagger to explore endpoints, send test requests, and view request/response schemas.

Endpoints

Method	Path	Description
--------	------	-------------

Example requests

Health

```
curl http://localhost:8080/health
```

RAG query

```
curl -X POST http://localhost:8080/rag/query `
```

Chat (with session)

```
curl -X POST http://localhost:8080/chat `
```

Assistant

```
curl -X POST http://localhost:8080/assistant `
```

Agent

```
curl -X POST http://localhost:8080/agent `
```

Ingest documents

```
curl -X POST http://localhost:8080/ingest
```

Prerequisites

Same as the main app:

1. uv sync --extra dev

2. Co

Project layout

api/

??? ma

Related docs

- * Health Check & Setup

* Ru

MiniGPT (merged project)

The MiniGPT project is integrated into this workspace under minigpt/. It complements the main stack (RAG, agents, MCP, API, Streamlit) with from-scratch GPT implementations and Llama 3 fine-tuning.

Location

AI_ML_GENAI/

??? ra

What MiniGPT includes

Module	Purpose
--------	---------

Quick start (PyTorch MiniGPT)

From repo root:

```
cd C:\MY_SPACE\AI_ML_GENAI
```

Expected chat:

You: twinkle

MiniG

TensorFlow MiniGPT

```
uv sync --extra minigpt-tensorflow
```

uv run

Llama 3 fine-tuning (finetune/)

Requires NVIDIA GPU + CUDA for QLoRA training.

```
uv sync --extra minigpt-finetune
```

cd min

Full manual: minigpt/finetune/COMPLETE_MANUAL.md

Optional dependency groups

Extra Command For

How MiniGPT fits the unified project

Capability Main project MiniGPT

Use MiniGPT to learn and experiment with model internals; use the main stack for production-style RAG, agents, and APIs.

Learning docs (inside minigpt/)

Topic Path

Original project

Merged from: C:\Users\nila0425\Downloads\MINIGPT

Related docs

* Models Guide

* RE